

PROBLEMA ESPECIAL

Sistemas Embebidos

Vidal Tomás, Chantiri Ignacio

https://github.com/TomiVidal99/SE_PE_1/



FACULTAD DE INGENIERÍA

EL PROBLEMA

El proyecto corresponde a un medidor de componentes R-C con autorango. Es decir, un sistema que sea capaz de medir valor de resistencia o capacidad de un DUT (*device under test*), y enviar dicha información por protocolo UART, para finalmente mostrarlo (en nuestro caso) en la pantalla de una PC, todo sin necesidad de ajustar manualmente el alcance de medición.

El sistema es configurable a dos modos de disparo:

- Continuo: Realiza una medición cada 100ms y actualiza constantemente la información en pantalla.
- Único: Realiza solo una medición y la muestra en pantalla.

Y por otro lado, es también configurable el parámetro a medir:

- Resistencia.
- Capacidad.

Dichas configuraciones se llevan a cabo desde el menú de inicio que se muestra y se maneja mediante comandos por UART desde la PC.

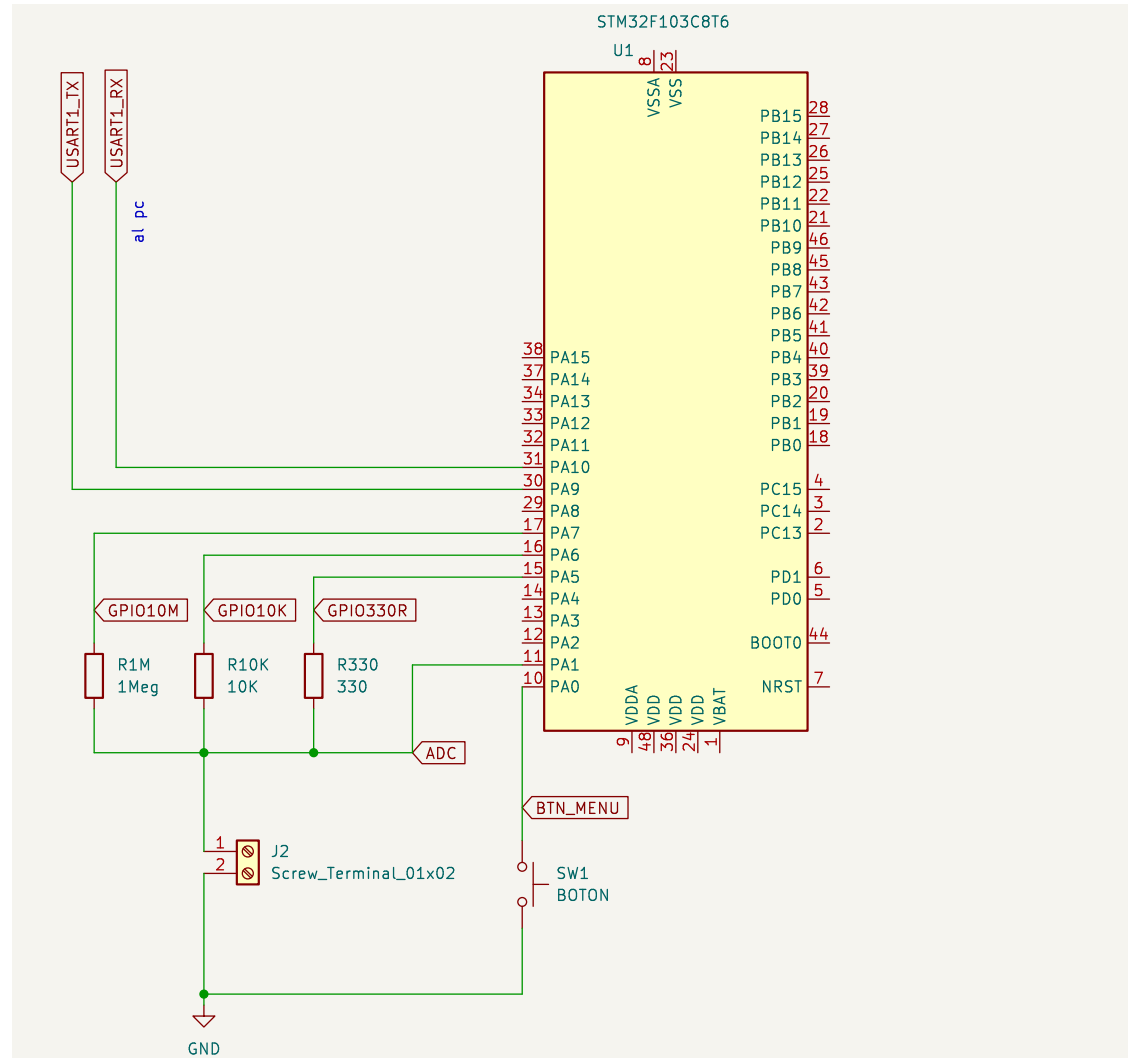
Una vez realizada la configuración, el inicio de medición se activa mediante la pulsación del único botón.

Para alternar en pantalla entre el menú de configuración y el menú que muestra el valor medido, se puede presionar el mismo botón.

Solución Hardware

La tarea completa se lleva a cabo con los siguientes materiales:

- Microcontrolador STM32F103C8T6
- 1 botón normalmente abierto
- 3 Resistencias de:
 - 330Ω
 - $10K\Omega$
 - $1M\Omega$
- Terminal para colocar el DUT



La función de autorango es necesaria para intentar minimizar el error de medición de resistencia. Según el circuito propuesto, el valor de resistencia se obtiene a partir del divisor resistivo:

$$R_{\text{DUT}} \approx R_{\text{ref}} \frac{V_{\text{med}}}{V_{\text{cc}} - V_{\text{med}}}$$

Graficando la función anterior con valores normalizados ($R_{\text{ref}} = 1$, $V_{\text{cc}} = 1$) y dejando V_{med} como variable x :



Donde se observa que por ser una relación no lineal, los valores de V_{med} cercanos a V_{cc} (x cercano a 1) provocan grandes cambios en el cálculo de R_{DUT} (y), por lo que el resultado de la medición será más sensible mientras más cerca esté la tensión medida de la de referencia. **Esto sucede si R_{med} es mucho más grande que R_{ref} , para el divisor resistivo en nuestro caso.**

El objetivo de diseño entonces se basa en esa premisa: **Intentar que el valor de resistencia a medir no sea demasiado grande respecto de la resistencia de referencia**, y se logra modificando R_{ref} entre tres opciones: 330, 10k y 1M, según se requiera.

La elección de la resistencia de referencia comienza en 330ohm. Si el valor de tensión medido está dentro del 5% del valor V_{cc} (3.27V), se switchea a la siguiente R_{ref} , la de 10k. Se repite la misma lógica para ver si se necesita cambiar a la última resistencia. En caso de que la última resistencia no sea suficiente para asegurar un error bajo de medición, se avisa con un mensaje de fuera de escala.

Descarga, carga y medida del capacitor

Previo a la medición, se debe realizar un **proceso de descarga** para asegurar el estado inicial del capacitor. Esto se logra Conectando el **DUT** directamente a 0V a través de la resistencia $1\text{M}\Omega$. *El dispositivo está listo para ser medido una vez que se obtiene con el ADC una tensión menor al 2% de V_{cc} (0.065V).*

Luego se procede a la carga, que se basa en la medición de la constante de tiempo τ de un circuito R-C, donde R es una resistencia R_{ref} conocida fija, de $1\text{M}\Omega$.

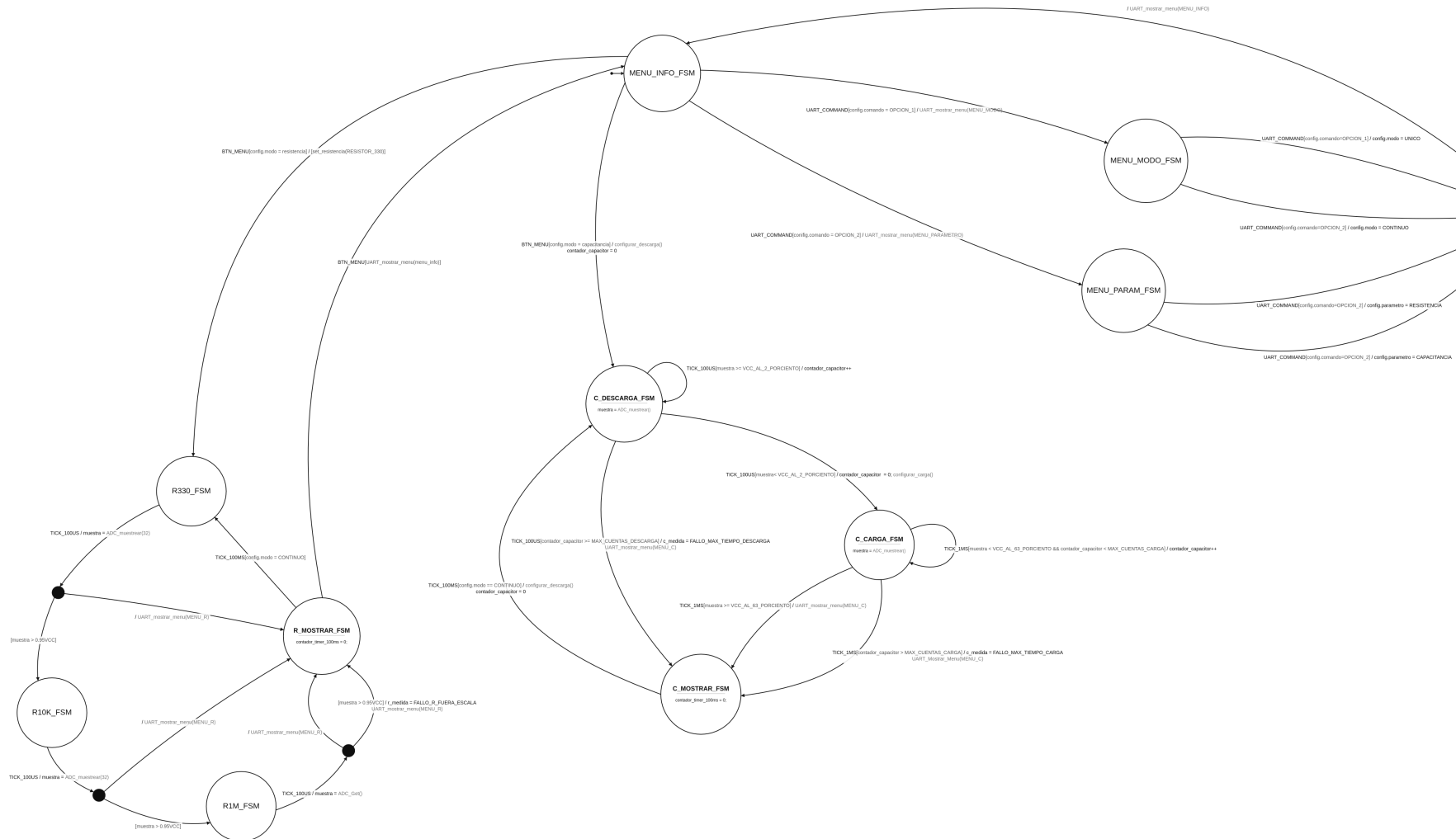
Encendiendo el GPIO correspondiente a $R_{\text{ref}} = 1\text{M}\Omega$ y el resto en alta impedancia, se forma entonces el circuito y comienza la carga del **DUT**.

$$C = \frac{\tau}{R_{\text{ref}}}$$

Para obtener el valor de τ , se cuentan muestras cada 1ms repetidamente hasta medir con el ADC una tensión igual al 63% del valor final de carga ($0.63V_{cc} = 2.06V$). El valor de capacidad en nano faradios será simplemente la cantidad de muestras, puesto que se asume que la R_{ref} es de exactamente $1M\Omega$ y también que el intervalo de muestreo es de precisamente 1ms.

Existe un timeout para el caso en que nunca se alcance el valor de carga anterior.
(fuera de escala)

Solución Software



- Se hizo una **Máquina de estados finitos (FSM)** para resolver el problema:
- Las señales de la **FSM** son:
- **TICK100US**: timer de 100us (generado por **TIM1**)
- **TICK1MS**: timer de 1ms (generado por contador a partir de **TIM1**)
- **TICK100MS**: timer de 100ms (generado por contador a partir de **TIM1**)
- **COMANDO**: Cuando el usuario ingresa un caracter por *UART* (**UART modo IT**)
- **BTN_MENU**: Cuando el usuario presiona el botón (**BOTÓN CON EXTI**)

Funcionamiento general de la **FSM**:

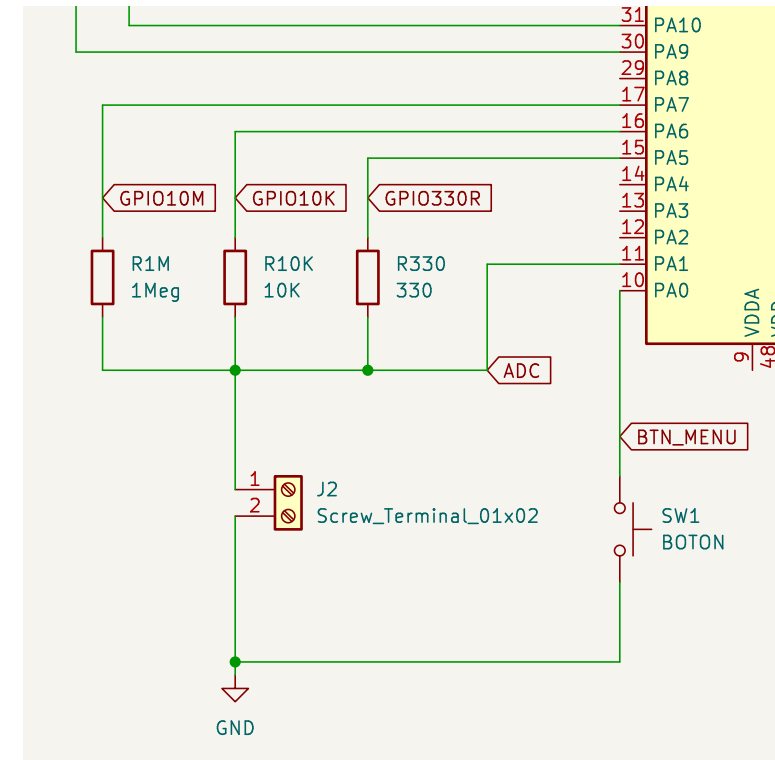
- Los estados: **R330_FSM**, **R10K_FSM**, **R1M_FSM** y **R_MOSTRAR_FSM**, se encargan de resolver la parte de medición de **resistencia**.
- Los estados: **C_DESCARGAR_FSM**, **C_CARGA_FSM** y **C_MOSTRAR_FSM**, se encargan de la parte de **capacidad**.
- Los estados: **MENU_INFO_FSM**, **MENU_MODO_FSM** y **MENU_PARAM_FSM**, se encargan de la parte del **menu UART**.

Funcionamiento general

Para los distintos tipos de medición se necesita conocer la tensión que cae en el DUT. Para ello, se conecta el ADC1 en sus terminales (como muestra el esquemático).

El método de muestreo elegido es mediante el cálculo de un promedio entre N muestras a la mayor frecuencia permitida: esto significa que apenas el ADC termina una conversión, se inicia la siguiente. El valor default para N es 32 pero es modificable por código (no por comando).

De acá en adelante, siempre que se mencione la



tensión medida V_{med} , nos referimos a la tensión medida promedio de las N muestras.

Librería de usuario

Multimetro.c contiene 2 funciones principales:

- **Multimetro_activar()**
- **Multimetro_procesar()**

```
/* USER CODE BEGIN 2 */

Multimetro_activar();

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1) {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
    Multimetro_procesar();

}
/* USER CODE END 3 */
}
```

Multimetro_activar()

- HAL_TIM_Base_Start_IT(): Comienza a correr **TIM1**.
- HAL_UART_Receive_IT(): Activa interrupciones del **UART**.
- UART_Mostrar_Menu(MENU_INFO): Muestra el menu inicial.

Multimetro_procesar()

- Actúa como el “*background*” de nuestra librería.
- Allí se capturan los flags de entrada y se llama a la **FSM** con el evento correspondiente:
- **flag_nuevo_comando**: Cada vez que llega una interrupción por **UART**.
- **btn_menu**: Cada vez que se presiona el **botón**.
- **TICKs**: Cada vez que se el **TIM1** dispara una interrupción cada *100us*, se procesan los contadores.

Menús de configuración

Partiendo desde el menú info, que indica la configuración actual (modo y parámetro), **es posible moverse entre los distintos submenús de configuración solamente con comandos por UART que se indican en pantalla.** Luego de la configuración de modo o de parámetro, siempre se vuelve al menú inicial (menú info).

Proceso de medición y menú medición

Cuando se llega al estado medir por medio de el botón, se procesa una primer medición. En caso de que el modo sea único, se procede a mostrar el menú medición con el valor obtenido. En caso de que el modo sea continuo, se comienza un bucle infinito medir-mostrar cada 100ms, hasta que se presione el botón una vez más para salir y volver al menú info.